

A5 Predicción de la demanda con AWS

Despliegue y visualización

Asignatura	Proyecto de inteligencia artificial y Big data
Autor	Adrián Blasco Lozano
Institución	Universitat Oberta de Catalunya
Fecha	11/05/2026
Repositorio	github.com/blasketch/prediccion-demanda-aws

1. Estrategia de despliegue

Esta actividad culmina el proyecto iniciado en R3, arquitectura cloud + pipeline ETL y continuado en R4, entrenamiento del modelo XGBoost. El objetivo de R5 es exponer el modelo entrenado como un servicio accesible y seguro que pueda consumirse desde sistemas externos o desde usuarios finales mediante una interfaz visual, completando así el ciclo MLOps del proyecto.

Sobre la infraestructura base de R3, Lambdas, API Gateway, DynamoDB, S3 con cifrado KMS, todo gestionado vía SAM, R5 incorpora cuatro piezas nuevas: un Lambda de inferencia real basado en Container Image que carga el modelo desde S3, autenticación obligatoria mediante API Key con Usage Plan, un dashboard interactivo en Streamlit como cliente visual de la API, y evidencias de observabilidad recogidas directamente desde CloudWatch.

1.1 Mecanismo de exposición elegido

Siguiendo la recomendación de elegir un solo mecanismo de exposición, he optado por API Gateway REST + Lambda como el mecanismo principal de exposición del modelo. El dashboard Streamlit se ha construido como un cliente que consume esta misma API, no como un segundo mecanismo paralelo. Esto cumple con el principio de claridad arquitectónica y maximiza la reutilización del trabajo previo: cualquier sistema puede consumir la misma API con el mismo contrato.

2. Arquitectura y configuración técnica

2.1 Diagrama de la arquitectura

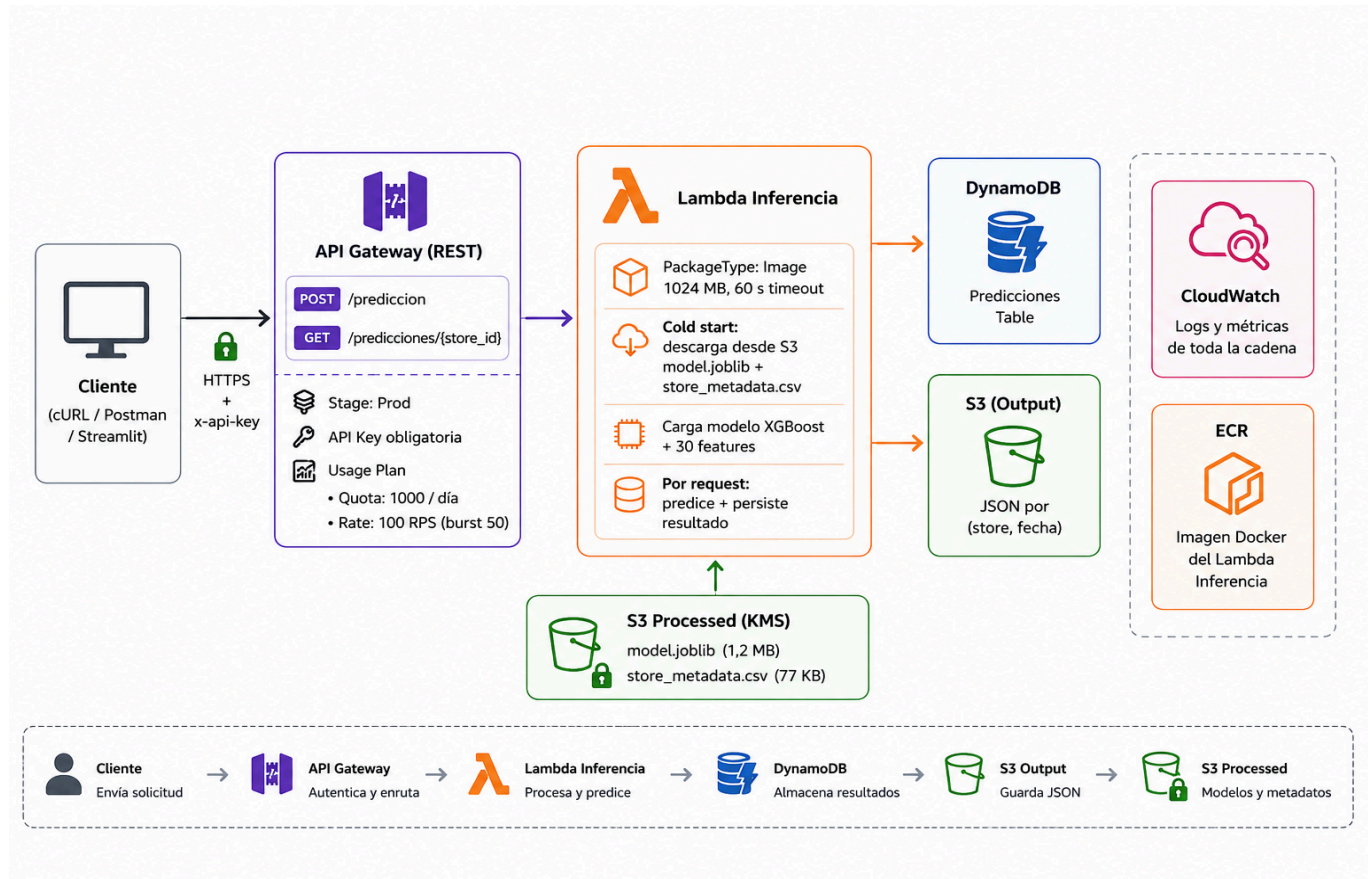


Figura 1. Diagrama esquemático de la arquitectura de despliegue R5.

2.2 Componentes principales y configuración

Lambda Inferencia

La función de inferencia se ha refactorizado completamente respecto a R3. En R3 quedó como esqueleto que llamaba a un endpoint SageMaker inexistente; en R5 se ha rediseñado para cargar el modelo joblib directamente desde S3 y predecir dentro del propio Lambda, simplificando la arquitectura.

Parámetro	Valor
Tipo de paquete	Container Image (Docker)
Imagen base	public.ecr.aws/lambda/python:3.12
Memoria	1024 MB (uso real medido: 296 MB)
Timeout	60 segundos
Dependencias clave	xgboost 3.2.0, scikit-learn 1.5.2, pandas 2.2.3, joblib 1.4.2
Variables de entorno	TABLE_NAME, OUTPUT_BUCKET, PROCESSED_BUCKET
Repositorio ECR	319501512128.dkr.ecr.us-east-1.amazonaws.com/predicciondemand ab9531b15/lambdainferencia13cb7076repo
Tracing	X-Ray activado

Tabla 1. Configuración técnica del Lambda Inferencia.

API Gateway

Parámetro	Valor
Tipo	REST, regional
Stage	Prod
Invoke URL	https://36bjw4fzt5.execute-api.us-east-1.amazonaws.com/Prod
Endpoints	POST /prediccion · GET /predicciones/{store_id}
Autenticación	API Key obligatoria (header x-api-key)
Usage Plan	1000 req/día · 100 RPS · burst 50
Trazas X-Ray	Activadas

Tabla 2. Configuración del API Gateway.

Recursos

Acciones de API ▼ Implementar API

[Crear recurso](#)

- /
- /prediccion POST
- /predicciones
- /store_id GET

/predicciones/{store_id} - GET - Ejecución de método Actualizar documentación Eliminar

ARN: `arn:aws:execute-api:us-east-1:319501512128:36bjw4fzt5/*/*/GET/predicciones/{store_id}` ID de recurso: `ptlok3`

Solicitud de método Solicitud de integración Integración de Lambda

Respuesta de método Respuesta de integración (integración de proxy)

Solicitud de método Solicitud de integración Respuesta de integración Respuesta de método Pruebas

Configuración de solicitud de método Editar

Autorización: NONE Clave de API obligatoria: Verdadero

Validador de solicitudes: Ninguna Nombre de la operación del SDK: Generado según el método y la ruta

Rutas de solicitud (1)

Nombre	Almacenamiento en caché
store_id	Inactivo

Figura 2. Consola de API Gateway mostrando los endpoints, el tipo de integración Lambda y la columna "Clave de API: Obligatorio" para ambos métodos.

Etapas

Acciones de etapa ▼ Create stage

Prod

- /
- Stage

Detalles de la etapa Información Editar

Nombre de etapa: Prod Tasa: 10000 ACL web: -

Clúster de caché: Inactivo Ampliación: 5000 Certificado de cliente: -

Almacenamiento en caché de nivel de método predeterminado: Inactivo

URL de invocación: `https://36bjw4fzt5.execute-api.us-east-1.amazonaws.com/Prod`

Figura 3. Stage Prod del API Gateway con el Invoke URL público desplegado.

DynamoDB, S3 y ECR

La tabla DynamoDB PrediccionesTable usa esquema StoreId + Fecha y modo PAY_PER_REQUEST. El cifrado en reposo es el gestionado por AWS por defecto. Los tres buckets S3 (raw, processed, output) están cifrados con SSE-KMS y tienen una BucketPolicy que rechaza explícitamente cualquier conexión que no use HTTPS. El repositorio ECR fue creado automáticamente por SAM al detectar PackageType: Image en la plantilla, gracias a la opción `resolve_image_repos = true` del `samconfig.toml`.

2.3 Infraestructura como código

Todo el stack está definido en un único fichero `infrastructure/template.yaml` (AWS SAM, basado en CloudFormation). El despliegue se realiza con dos comandos:

```
sam build
```

```
sam deploy
```

SAM gestiona automáticamente la construcción de la imagen Docker, su publicación en ECR, la creación del `changeset` CloudFormation y el despliegue secuencial de los recursos respetando las dependencias entre ellos.

2.4 Escalado

Componente	Mecanismo de escalado	Límites configurados
API Gateway	Account-level: 10.000 RPS regional default	100 RPS / 50 burst en Usage Plan
Lambda Inferencia	Concurrencia automática	1000 concurrent (account)
DynamoDB	On-demand auto-scaling	Sin límite manual
S3	Auto-scaling transparente	N/A

Tabla 3. Mecanismos de escalado configurados por servicio.

2.5 Tiempo de respuesta medido

Tras un primer cold start del contenedor, las invocaciones warm devuelven respuesta en 47–210 ms, perfectamente apto para uso interactivo desde el dashboard. El cold start completo, la descarga del modelo de S3 + imports de xgboost/sklearn, ronda los 10 o 15 segundos, con picos puntuales de hasta 34 segundos en la primera invocación absoluta tras un nuevo despliegue.

3. Seguridad y accesibilidad

3.1 Capas de seguridad implementadas

El sistema implementa seguridad en defensa en profundidad, con controles en cada capa de la arquitectura:

Capa	Implementación	Evidencia
Cifrado en tránsito	HTTPS obligatorio en API Gateway. BucketPolicy con Deny SecureTransport=false en S3	template.yaml (BucketPolicy)
Cifrado en reposo	S3 con SSE-KMS (alias aws/s3), DynamoDB cifrado por defecto	template.yaml (BucketEncryption)
Autenticación	API Key obligatoria (header x-api-key)	OpenAPI: security: [api_key: []] en cada path
Autorización	LabRole asignado a cada Lambda	Role: arn:aws:iam::319501512128:r ole/LabRole
Rate limiting	Usage Plan: 1000 req/día, 100 RPS, burst 50	Captura figura 4
Validación de entrada	Lambda valida store_id contra metadata; devuelve 400 si no existe	Test 5 en results/api_tests/

Tabla 4. Capas de seguridad implementadas en el sistema.

3.2 IAM y principio de mínimo privilegio

En el entorno AWS Academy Learner Lab no es posible crear roles IAM personalizados; todas las funciones Lambda usan el rol genérico LabRole, que dispone de permisos amplios sobre los servicios habilitados.

En un entorno productivo real, la aplicación del principio de mínimo privilegio se haría desplegando dos roles IAM personalizados, uno por Lambda.

Esto eliminaría privilegios innecesarios sobre el resto de servicios del account y reduciría el blast radius en caso de compromiso de credenciales del Lambda.

3.3 Verificación funcional de la seguridad

Se han ejecutado dos pruebas contrastadas para validar empíricamente la efectividad de la autenticación por API Key, los resultados completos estan en results/api_tests/:

Llamada POST /prediccion SIN header x-api-key:

```
$ curl -X POST -H "Content-Type: application/json" \
  -d '{"store_id": 1, "fecha": "2015-09-17", "promo": 1}' \
  https://36bjw4fzt5.execute-api.us-east-1.amazonaws.com/Prod/prediccion
```

Respuesta: {"message":"Forbidden"} (HTTP 403)

Llamada POST /prediccion CON header válido:

```
$ curl -X POST -H "Content-Type: application/json" \
  -H "x-api-key: $API_KEY" \
  -d '{"store_id": 1, "fecha": "2015-09-17", "promo": 1}' \
  https://36bjw4fzt5.execute-api.us-east-1.amazonaws.com/Prod/prediccion
```

Respuesta: {"store_id": 1, "fecha": "2015-09-17",
 "prediccion_unidades": 4667.26,
 "modelo": "xgboost-r4-final",
 "rmspe_test": 0.1483}

Ambas pruebas confirman que la API rechaza correctamente las llamadas no autenticadas y atiende sólo las llamadas con la API Key válida en el header. La validación se complementa con el rate limiting del Usage Plan, que se evidencia en la siguiente figura.

prediccion-demanda-usage-plan Última actualización: May 11, 2026, 05:53 (UTC+02:00) [Acciones](#) [Exportar datos de uso](#)

Detalles del plan de uso

ID del plan de uso tt8lvj	Tasa 100 solicitudes por segundo
Descripción Plan de uso para la API de predicción de demanda	Ampliación 50 solicitudes
Código de producto de AWS Marketplace -	Cuota 1000 solicitudes por día

Etapas asociadas | Claves de API asociadas | Etiquetas

Etapas asociadas (1) [Información](#) [Editar](#) [Eliminar](#) [Agregar etapa](#)

API	Etapas	Limitación de método
prediccion-demanda-APIGateway	Prod	-

Figura 4. Usage Plan en API Gateway con cuota de 1000 requests por día y limitación de 100 RPS con burst 50.

4. Pruebas funcionales

Se han ejecutado 8 pruebas que cubren el flujo completo de la API, la validación de inputs y la demostración de la persistencia en DynamoDB. Cada test está documentado individualmente en `results/api_tests/` con sus archivos de entrada y salida.

#	Endpoint	Input clave	Output
1	POST /prediccion	Store 1, 2015-09-17, Promo=1	4 667,26 €
2	POST /prediccion	Store 1, 2015-09-17, Promo=0	3 740,83 €
3	POST /prediccion	Store 100, 2015-09-17, Promo=1	7 421,96 €
4	GET /predicciones/1		Histórico DynamoDB
5	POST /prediccion	Store 99999 (inválido)	{"error": "Store 99999 no existe"}
6	POST sin API Key		{"message": "Forbidden"} (403)

7	POST con API Key	Store 1, Promo=1	Predicción 200 OK
8	GET con API Key	Store 1	Histórico DynamoDB

Tabla 5. Resumen de las 8 pruebas funcionales ejecutadas sobre el sistema desplegado.

4.1 Análisis de la coherencia del modelo

Los resultados de los tests 1, 2 y 3 demuestran que el modelo XGBoost desplegado responde correctamente a las variaciones de entrada:

- **Tests 1 vs 2:** para la misma tienda y fecha, activar la promoción incrementa la predicción de 3.740,83 € a 4.667,26 €. Esto valida que el modelo ha aprendido el efecto de las promociones sobre las ventas, coherente con el análisis de R4 donde se observaba que los días con promo tenían RMSPE significativamente mejor.
- **Tests 1 vs 3:** para la misma fecha y misma activación de promoción, la tienda 100 predice 7.421,96 € frente a los 4.667,26 € de la tienda 1. Esto confirma que el lookup de store_metadata.csv funciona correctamente, devolviendo features estáticas distintas por tienda.
- **Test 4:** la consulta histórica devuelve la última predicción almacenada en DynamoDB para esa tienda. Si se hacen múltiples predicciones para la misma, el ítem se sobrescribe, comportamiento correcto.
- **Test 5:** el Lambda valida que el store_id exista en el conjunto de tiendas conocidas y devuelve un error explícito 400 si no es así. Esto previene predicciones sobre identificadores inexistentes.

5. Visualización: dashboard Streamlit

Se ha desarrollado un dashboard local en Streamlit como cliente visual de la API REST. El código está en streamlit/dashboard.py y consume la API mediante la librería requests, inyectando la API Key vía st.secrets.

5.1 Funcionalidad implementada

El dashboard expone dos pestañas:

Pestaña "Nueva predicción":

Formulario con los inputs que cambian en el tiempo: store_id (selector numérico 1-1115), fecha (date picker), Promo (toggle), state_holiday (selector:

Ninguno/Público/Pascua/Navidad), y school_holiday (toggle). Al pulsar el botón "Predecir" se llama a la API y se muestra el resultado con un panel de métricas destacado (ventas predichas en €, tienda, fecha) más un expander con el request/response completos en JSON, útil para depuración y para la demo técnica.

Predicción de Demanda — Rossmann Stores

Sistema de predicción de ventas diarias por tienda.
Modelo: XGBoost (RMSPE 14.83 % en test) · Arquitectura: API Gateway + Lambda + DynamoDB

[Nueva predicción](#) [Historio por tienda](#)

Solicitar predicción

ID de Tienda: ⓘ ⊖ ⊕ Promoción activa ☒

Fecha objetivo: ⓘ Festivo estatal Ninguno ⌵

☐ Vacaciones escolares

⌂ Predecir

✓ Predicción realizada

Ventas predichas	Tienda	Fecha
4,667.26 €	#1	2015-09-17

Modelo: xgboost-r4-final · RMSPE test: 14.83 %

⌵ Ver request / response completos

```
{
  "request": {
    "store_id": 1
    "fecha": "2015-09-17"
    "promo": 1
    "state_holiday": 0
    "school_holiday": 0
  }
  "response": {
    "store_id": 1
    "fecha": "2015-09-17"
    "prediccion_unidades": 4667.26
    "modelo": "xgboost-r4-final"
    "raspe_test": 0.1483
  }
}
```

Figura 5. Pestaña "Nueva predicción" del dashboard Streamlit tras ejecutar una predicción exitosa para Store 1 con promoción activa.

Pestaña "Histórico por tienda":

Selector de store_id y botón de consulta que invoca GET /predicciones/{store_id}, devolviendo las predicciones almacenadas en DynamoDB para esa tienda. Los resultados se muestran como tabla ordenada por timestamp descendente, con las columnas "Fecha predicción", "Ventas predichas (€)" y "Generado en".

Predicción de Demanda — Rossmann Stores

Sistema de predicción de ventas diarias por tienda.
Modelo: XGBoost (RMSPE 14.83 % en test) · Arquitectura: API Gateway + Lambda + DynamoDB

[Nueva predicción](#) [Historio por tienda](#)

Histórico de predicciones desde DynamoDB

ID de Tienda: ⓘ ⊖ ⊕ Consultar

1 predicción(es) encontrada(s)

Fecha predicción	Ventas predichas (€)	Generado en
2015-09-17	4667.26	2025-05-11T03:29:23.153007

Adrián Blasco Lozano · UOC · Reto 5: Despliegue y visualización

Figura 6. Pestaña "Histórico por tienda" del dashboard Streamlit consultando las predicciones almacenadas en DynamoDB para la Store 1.

Sidebar informativa:

Contiene un resumen del modelo desplegado, el algoritmo, hiperparámetros, RMSPE 14,83 %. También el detalle de la infraestructura AWS (Lambda, API Gateway, DynamoDB, S3, CloudWatch) y las capas de seguridad implementadas.

5.2 Ejecución local

El dashboard se ejecuta localmente con los siguientes pasos, documentados en streamlit/README.md:

```
$ cd streamlit
$ python -m venv .venv
$ source .venv/bin/activate
$ pip install -r requirements.txt
$ cp .streamlit/secrets.toml.example .streamlit/secrets.toml
# Editar secrets.toml con los valores reales de API_URL y API_KEY
$ streamlit run dashboard.py
```

Streamlit abre automáticamente la interfaz en <http://localhost:8501>. La elección de ejecución local en lugar de despliegue en Streamlit Cloud o EC2 obedece a tres criterios:

- El dashboard es una herramienta interna del proyecto académico
- Se evita coste adicional de cómputo y configuración de infraestructura
- La API_KEY se mantiene en el sistema local.

5.3 Demo en vídeo

Junto a esta memoria se adjunta el archivo demo_R5.mp4, una grabación de pantalla de aproximadamente 2-3 minutos que muestra el flujo completo del sistema funcionando, invocación de la API vía cURL (con y sin API Key), predicción y consulta de histórico desde el dashboard Streamlit, y visualización de los logs y métricas en CloudWatch.

6. Observabilidad y métricas

6.1 CloudWatch logs

El Lambda Inferencia implementa logging estructurado con módulo logging de Python, registrando cada fase del cold start y cada predicción con el RequestId asociado para trazabilidad. Los logs van automáticamente a CloudWatch Logs en el grupo `/aws/lambda/prediccion-demanda-LambdaInferencia-R4RUpsndnGNp`

Ejemplo real de traza de una invocación cold start:

INFO Cold start: descargando model.joblib desde S3

INFO Modelo cargado | Features: 30 | RMSPE test: 0.1483

INFO Cold start: descargando store_metadata.csv desde S3

INFO Metadata de 1115 tiendas cargada

INFO Store 1 | 2015-09-17 | DOW 4 | Promo 1 → 4667.26€

REPORT Duration: 34272.75 ms | Memory: 1024 MB | Max Memory: 296 MB

CloudWatch <

Favoritos y recientes ▶

Ingesta

Paneles

Alarmas

► Operaciones de inteligencia artificial

► Observabilidad de GenAI

► Señales de aplicación (APM) [Nuevo](#)

► Monitorización de la infraestructura

▼ Registros

[Administración de registros](#)

[Análisis de registros](#)

Eventos de registro

Puede utilizar la barra de filtros a continuación para buscar y hacer coincidir términos, frases o valores en sus eventos de registro. [Más información sobre los patrones de filtro](#)

Q Filtrar eventos: pulse Intro para buscar

►	Marca temporal	Mensaje
		No hay eventos antiguos en este momento. Volver a intentar
►	2026-05-11T03:26:14.110Z	/var/task/joblib/_multiprocessing_helpers.py:46: UserWarning: [Errno 2] No such file or directory. joblib will operate in serial mode
►	2026-05-11T03:26:14.110Z	warnings.warn("%s. joblib will operate in serial mode" % (e,))
►	2026-05-11T03:26:14.715Z	[INFO] 2026-05-11T03:26:14.715Z Found credentials in environment variables.
►	2026-05-11T03:26:14.892Z	START RequestId: bf56cc84-eb25-4279-8534-5ccaa8ae93e Version: \$LATEST
►	2026-05-11T03:26:14.893Z	[INFO] 2026-05-11T03:26:14.893Z bf56cc84-eb25-4279-8534-5ccaa8ae93e Cold start: descargando model.joblib desde S3
►	2026-05-11T03:26:18.012Z	[INFO] 2026-05-11T03:26:18.012Z bf56cc84-eb25-4279-8534-5ccaa8ae93e Modelo cargado Features: 30 RMSPE test: 0.1483
►	2026-05-11T03:26:18.012Z	[INFO] 2026-05-11T03:26:18.012Z bf56cc84-eb25-4279-8534-5ccaa8ae93e Cold start: descargando store_metadata.csv desde S3
►	2026-05-11T03:26:18.116Z	[INFO] 2026-05-11T03:26:18.116Z bf56cc84-eb25-4279-8534-5ccaa8ae93e Metadata de 1115 tiendas cargada
►	2026-05-11T03:26:18.132Z	[INFO] 2026-05-11T03:26:18.132Z bf56cc84-eb25-4279-8534-5ccaa8ae93e Store 1 2015-09-17 DOW 4 Promo 1 → 4667.26€

Figura 7. CloudWatch Logs del Lambda Inferencia mostrando las trazas reales del cold start y de las predicciones procesadas.

6.2 CloudWatch metrics

Las métricas de invocación se recogen automáticamente sin código adicional. Datos extraídos directamente del API CloudWatch:

Bucket UTC	Invocaciones	Duración media	Duración máxima	Errores
03:22:00	3	1.643 ms	4.778 ms	0
03:27:00	2	95 ms	180 ms	0
03:32:00	1	47 ms	47 ms	0

Tabla 6. Métricas CloudWatch del Lambda Inferencia.

Interpretación de los datos:

- El bucket de las 03:22 capturó la primera invocación tras el deploy (cold start completo del contenedor incluyendo descarga del modelo desde S3 e imports pesados de xgboost/sklearn) más dos llamadas subsiguientes warm, dando una media inflada de 1,6 s y un pico de 4,8 s.
- Los buckets posteriores muestran latencia warm estable de 47-180 ms, perfectamente apta para uso interactivo.
- 0 errores en toda la ventana, validando la robustez del Lambda y la integración API Gateway Lambda.
- Memoria utilizada: 296 MB de 1024 MB asignados. En una iteración posterior se podría reducir la asignación de memoria a 512 MB para optimizar coste sin afectar al rendimiento.

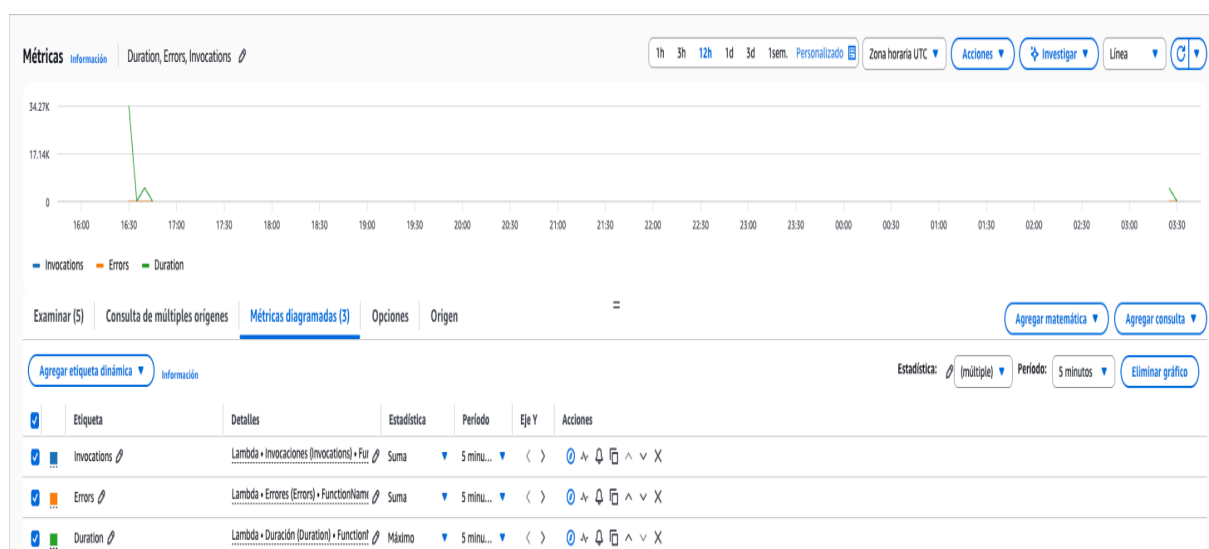


Figura 8. Gráfica de CloudWatch Metrics mostrando *Invocations (Sum)*, *Duration (Average + Maximum)* y *Errors (Sum)* en una ventana de 2 horas.

6.3 Persistencia: tabla DynamoDB

Cada predicción se almacena en la tabla `PrediccionesTable` con esquema `StoreId (HASH) + Fecha (RANGE)`, atributos `Prediccion` y `Timestamp`. Esto permite consultar el histórico de predicciones por tienda y soporta upserts naturales (mismo store + fecha = se sobrescribe).

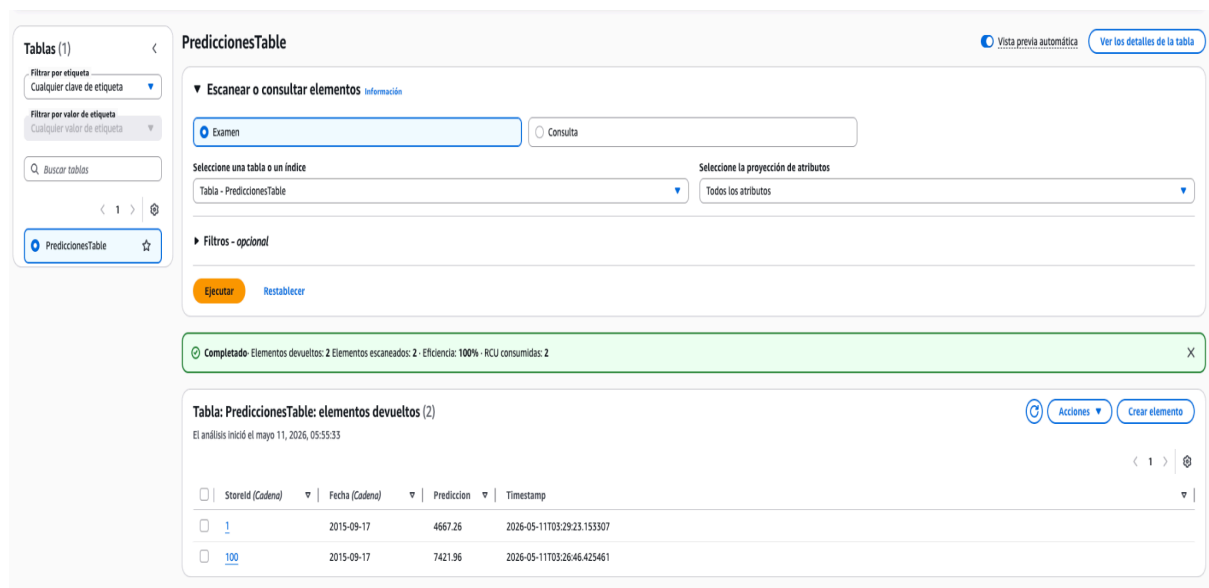


Figura 9. Items en la tabla `PrediccionesTable` de DynamoDB tras ejecutar las pruebas funcionales.

6.4 ARNs y recursos desplegados

La tabla completa de los 21 recursos AWS del stack está en `docs/r5_recursos_aws.txt`. Los principales son:

Recurso	Identificador / ARN
API Gateway (REST API)	36bjw4fzt5
API Gateway (Stage)	Prod
Lambda Inferencia	prediccion-demanda-LambdaInferencia-R4RUpSndnGNp
Lambda ETL	prediccion-demanda-LambdaETL
API Key (header x-api-key)	z5xaw6tt1i
Usage Plan	tt8ivj

DynamoDB Table	PrediccionesTable
S3 Bucket (raw)	prediccion-demanda-raw-319501512128
S3 Bucket (processed)	prediccion-demanda-processed-319501512128
S3 Bucket (output)	prediccion-demanda-output-319501512128
ECR Repository	predicciondemanda9531b15/lambdainferencia13cb7076repo

Tabla 7. Identificadores de los recursos AWS principales desplegados por el stack.

7. Reproducibilidad

Todo el código, la infraestructura como código y las evidencias están versionados en GitHub. El repositorio sigue una estrategia de branching por actividad, con esta entrega en la rama r5.

Repositorio: github.com/blasketch/prediccion-demanda-aws

Rama de esta actividad: r5

7.1 Despliegue desde cero en cualquier cuenta AWS

```
$ git clone https://github.com/blasketch/prediccion-demanda-aws.git
```

```
$ cd prediccion-demanda-aws
```

```
$ git checkout r5
```

```
# Paso 1: generar store_metadata.csv
```

```
$ python3 scripts/build_store_metadata.py
```

```
# Paso 2: build + deploy de la infraestructura completa
```

```
$ cd infrastructure
```

```
$ sam build
```

```
$ sam deploy
```


SAM gestiona automáticamente la creación del repositorio ECR (gracias a `resolve_image_repos = true`), la construcción y publicación de la imagen Docker, y el despliegue de todos los recursos via CloudFormation.

8. Limitaciones y mejoras futuras

He identificado varias limitaciones que en un entorno productivo real merecerían atención:

Cold start de 10-15 segundos.

Es aceptable para uso humano interactivo desde el dashboard, pero penaliza llamadas automatizadas esporádicas. En producción se mitigaría configurando Provisioned Concurrency en el Lambda o con un mecanismo de calentamiento programado (EventBridge invocando el Lambda cada 5 minutos).

API Key sin rotación automática.

La API Key actual es estática. En producción se implementaría rotación periódica, mensual por ejemplo, con AWS Secrets Manager o con un mecanismo basado en un script Lambda que genere nuevas keys, las propague a los consumidores autorizados y deprecie las antiguas tras un periodo de gracia.

Falta de WAF en el frontend de la API.

En producción, API Gateway se protegería detrás de AWS WAF para bloquear ataques comunes (SQLi en parámetros, IPs maliciosas conocidas, geo-blocking si aplica, rate limits adicionales por IP).

Modelo monolítico.

El análisis de errores de R4 identificó que aproximadamente 10 tiendas concentran el 30 % del error total. Una mejora natural sería clusterizar las tiendas por patrón de ventas (StoreType × Assortment × volumen) y entrenar modelos especializados por cluster, sirviéndolos todos desde el mismo Lambda con un selector de modelo por tienda.

9. Conclusiones

La actividad R5 culmina la cadena MLOps del proyecto, integrando un sistema de inferencia real, accesible vía API REST, autenticado, observable y con visualización interactiva. La arquitectura elegida (API Gateway + Lambda Container Image + Streamlit cliente) maximiza

el reaprovechamiento de la infraestructura de R3, respeta las restricciones del entorno Learner Lab y sigue patrones serverless estándar de la industria para inferencia ML.

Las métricas reales obtenidas durante las pruebas confirman que el sistema es funcional y robusto: cero errores en todas las invocaciones de test, latencia warm de 47 ms, la cual es apta para uso interactivo desde el dashboard, y validación end-to-end del flujo predicción - DynamoDB - consulta histórica. La seguridad implementada (HTTPS obligatorio, API Key, Usage Plan, cifrado KMS at-rest, validación de entradas) cubre las capas estándar de protección, dejando documentado el camino hacia una IAM granular en un entorno productivo real.

El uso de Container Image para el Lambda ha sido la decisión técnica más relevante de esta actividad: elimina las limitaciones de tamaño del paquete zip de Lambda, permite gestionar dependencias ML pesadas vía Dockerfile reproducible y se integra de forma nativa con SAM. El coste pagado es un cold start algo más largo, pero la latencia warm es excelente y el sistema se mantiene completamente serverless.

Esta actividad me ha permitido consolidar las prácticas de MLOps modernas: desplegar modelos ML como servicios serverless reproducibles desde IaC, exponerlos de forma segura con autenticación y rate limiting, y observarlos mediante telemetría estándar. El proyecto queda en un estado deployable desde cero en cualquier cuenta AWS mediante git clone + sam deploy.